Patrones de Diseño - IS

Diagrama UML

- Realizar el diagrama UML de clases correspondiente a la implementación.
- Representar las clases, atributos, métodos y relaciones principales del patrón.

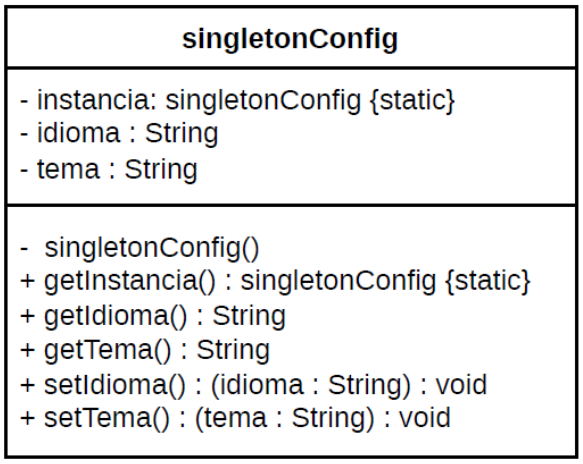
Problema – Solución – Consecuencias

- Explicar qué problema de diseño busca solucionar.
- Explicar cómo el patrón resuelve ese problema.
- Indicar las principales consecuencias, ventajas y posibles desventajas de utilizarlo.

Creational (Creacionales):

Singleton

Diagrama UML



Problema

En muchas aplicaciones existen recursos o componentes centralizados (como un gestor de configuraciones, un registro de logs o una conexión a una base de datos) donde permitir múltiples instancias genera inconsistencias de datos, uso innecesario de memoria o

conflictos de concurrencia. Se necesita garantizar que una clase tenga una y una sola instancia, y proveer un acceso global a ella.

Solución

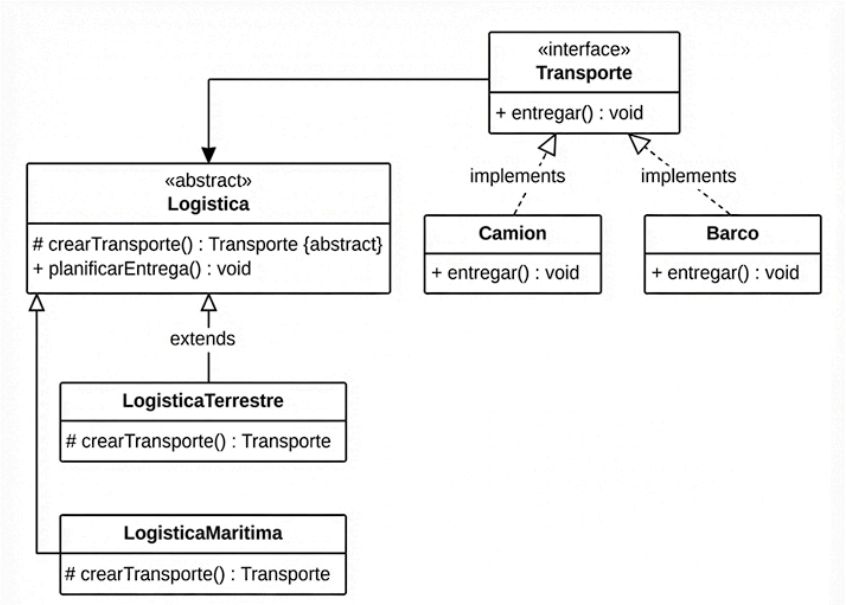
El patrón restringe la creación de objetos haciendo que el constructor sea privado. Obliga a que la propia clase gestione su única instancia a través de un método estático controlado (getInstance), el cual evalúa si ya fue creada antes de retornarla.

Consecuencias

- **Ventajas:**
    - Evita que existan múltiples instancias de la clase.
    - Proporciona un único punto de acceso a la instancia.
    - La creación de la instancia puede controlarse desde la propia clase.
- **Desventajas / Implicancias:**
    - Introduce un acceso global a la instancia.
    - Puede generar un acoplamiento fuerte entre las clases que utilizan el Singleton y esa clase.
    - Su uso excesivo puede dificultar el mantenimiento y las pruebas del sistema.

Factory Method

Diagrama UML



Problema

Si creas objetos directamente usando el operador new en tu código principal (por ejemplo, new Camión()), tu sistema queda fuertemente acoplado. Si mañana necesitas agregar un

nuevo transporte (como un avión), tendrías que modificar el código existente en varios lugares, lo que rompe las buenas prácticas y hace que el mantenimiento sea difícil y propenso a errores.

Solución

El patrón Factory Method propone delegar la creación de objetos a un método especializado (la "fábrica") dentro de clases abstractas.

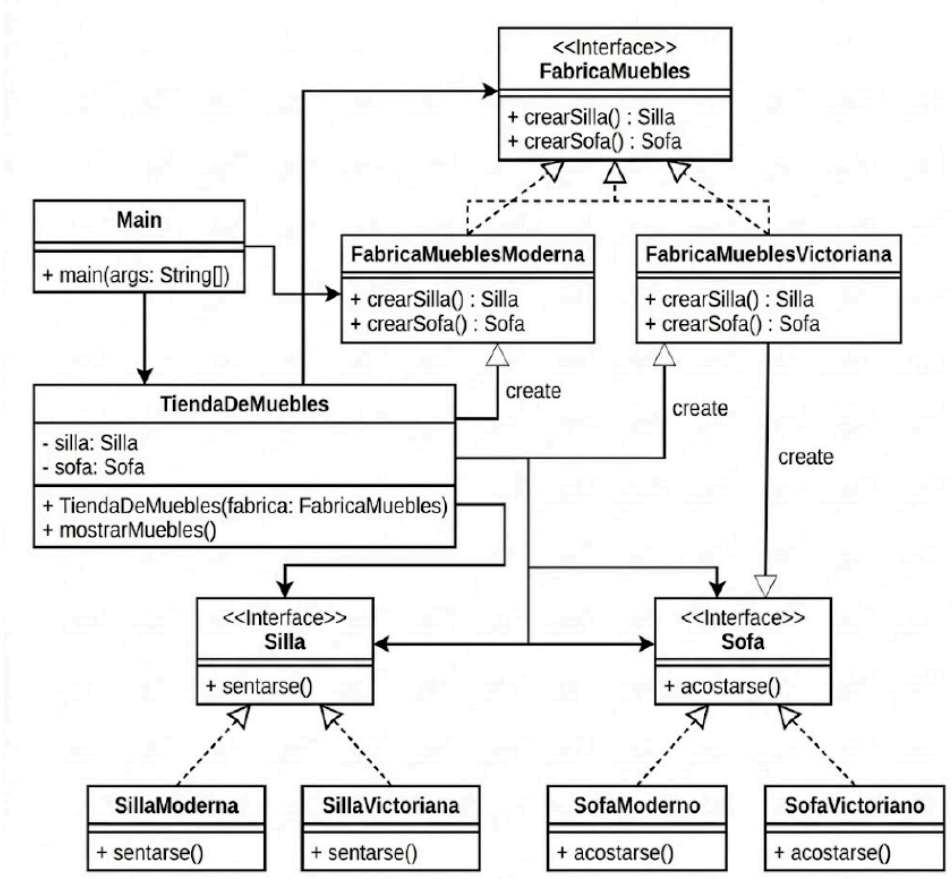
- En lugar de instanciar un camión directamente, la lógica de negocio llama a un método abstracto crearTransporte().
- Las subclases (LogisticaTerrestre, LogisticaMaritima) deciden qué objeto específico (Camión o Barco) crear. De esta forma, la lógica principal funciona con interfaces generales (Transporte, Logistica) sin importarle qué clase concreta está operando.

Consecuencias

- **Ventajas:**
  - Evita el acoplamiento directo entre clases.
  - Facilita la escalabilidad: puedes agregar nuevos transportes creando nuevas clases sin tocar el código que ya funciona (Principio Abierto/Cerrado).
  - El código de creación se centraliza en un solo lugar.
- **Desventajas:**
  - El proyecto se vuelve más complejo porque obliga a crear muchas clases pequeñas e interfaces adicionales.

Abstract Factory

Diagrama UML



Problema

El problema aparece cuando necesitamos crear varios objetos relacionados entre sí, pero queremos que todos pertenezcan a la misma familia o estilo.

En este caso, la tienda necesita crear muebles como Silla y Sofa, pero existen diferentes estilos: moderno y victoriano. Si la tienda tuviera que decidir directamente qué clase concreta crear (SillaModerna, SofaModerno, etc.), quedaría acoplada a esas clases y sería más difícil agregar nuevos estilos.
Además, existe el riesgo de combinar productos de diferentes familias, por ejemplo, crear una SillaModerna junto con un SofaVictoriano.

Solución

Abstract Factory propone crear una fábrica abstracta que define los métodos necesarios para crear cada producto de una familia.
En este caso, FabricaMuebles define:

- crearSilla()
- crearSofa()

Luego, cada fábrica concreta crea una familia completa de productos:

- FabricaMueblesModerna → SillaModerna y SofaModerno.

- FabricaMueblesVictoriana → SillaVictoriana y SofaVictoriano.

La clase TiendaDeMuebles trabaja solamente con las interfaces FabricaMuebles, Silla y Sofa. Por lo tanto, no necesita conocer las clases concretas.
De esta manera, al cambiar la fábrica que recibe la tienda, se puede cambiar toda la familia de muebles sin modificar la lógica de la tienda.

Consecuencias

Ventajas:

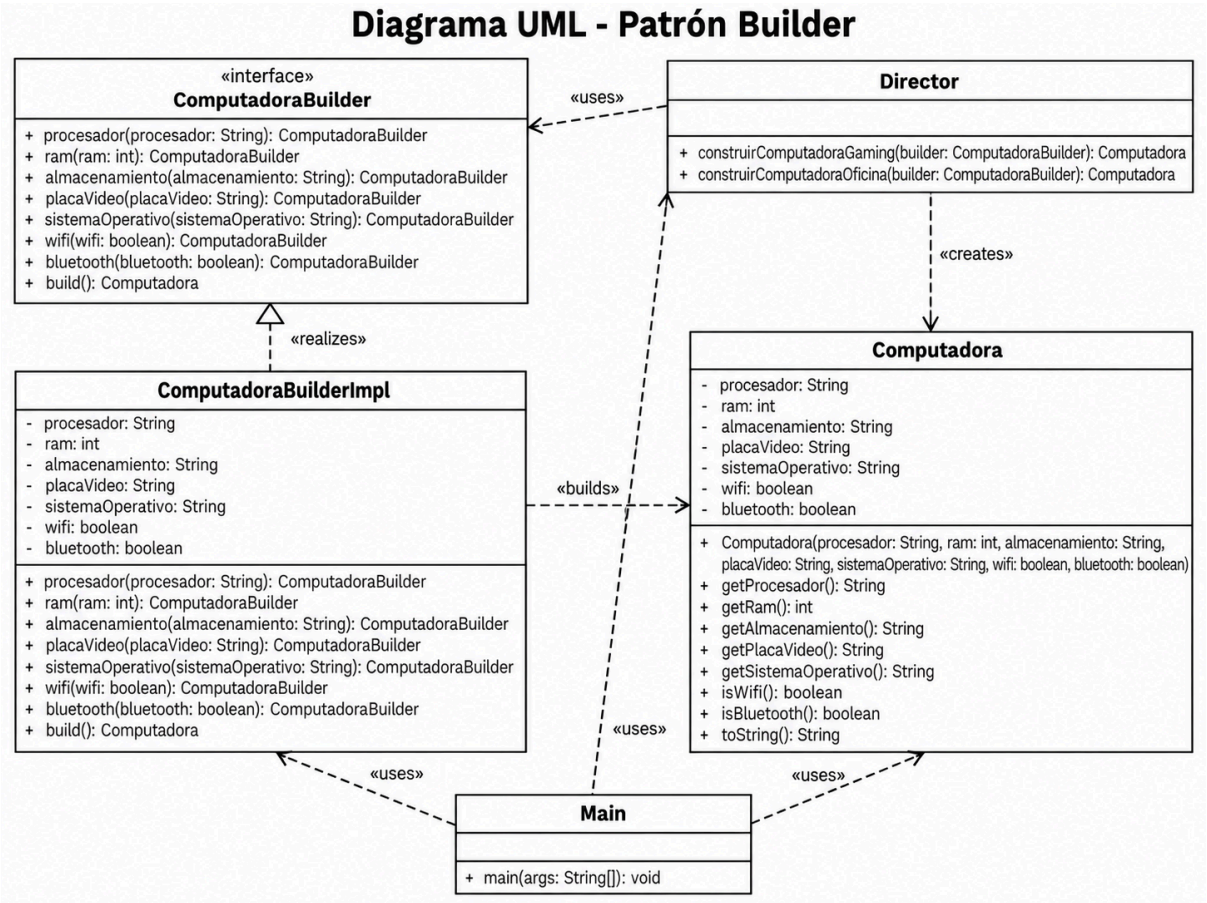
- Permite crear familias completas de objetos relacionados.
- Evita mezclar productos de diferentes familias.
- Reduce el acoplamiento entre el cliente y las clases concretas.
- Facilita agregar una nueva familia de productos sin modificar el cliente.
- Centraliza la creación de los objetos en las fábricas concretas.

Desventajas:

- Aumenta la cantidad de clases e interfaces necesarias.
- Agregar un nuevo tipo de producto, por ejemplo Mesa, requiere modificar la fábrica abstracta y todas las fábricas concretas.
- Puede resultar más complejo que crear directamente los objetos cuando el sistema es pequeño.

Builder

Diagrama UML



En esta implementación participan las siguientes clases:

- **Computadora:** es el producto que se desea construir.
- **ComputadoraBuilder:** interfaz que define los pasos necesarios para configurar una computadora.
- **ComputadoraBuilderImpl:** Builder concreto que implementa los pasos.
- **Director:** define configuraciones predeterminadas de computadoras.
- **Main:** utiliza el Director y el Builder para crear los objetos.

El **`ComputadoraBuilder`** posee métodos para configurar procesador, RAM, almacenamiento, placa de video, sistema operativo, WiFi y Bluetooth, finalizando con el método `build()`.

Problema

El patrón Builder busca solucionar el problema de crear objetos complejos que poseen muchos atributos o diferentes configuraciones posibles.

Sin este patrón, sería necesario utilizar constructores con una gran cantidad de parámetros.

Solución

Builder separa el proceso de construcción del objeto del objeto final.

En lugar de enviar todos los valores directamente al constructor, se utiliza un **Builder** que permite establecer los atributos paso a paso. Ejemplo:

builder

.procesador("Intel i7")

.ram(32)

.almacenamiento("1TB SSD")

.placaVideo("RTX 4070")

.sistemaOperativo("Windows 11")

.wifi(true)

.bluetooth(true)

.build();

Consecuencias

Ventajas

- Mayor legibilidad

- Construcción paso a paso

- Permite diferentes configuraciones

- Separa la construcción del producto

- Facilita la reutilización

- Permite crear objetos inmutables

Desventajas

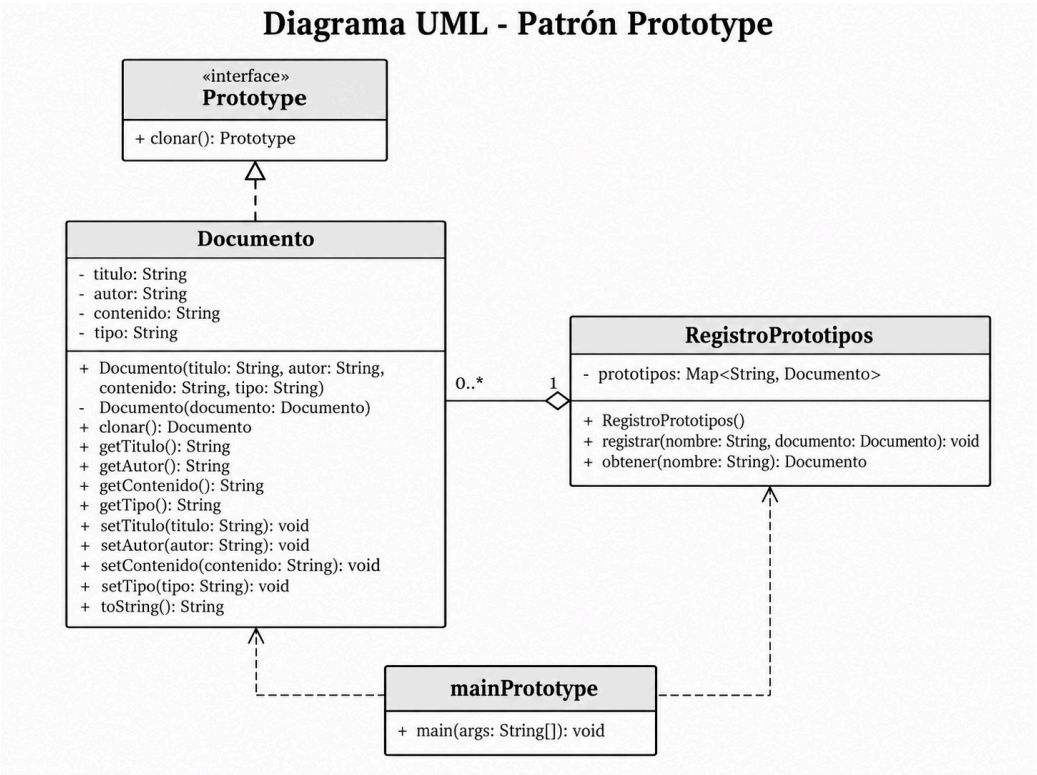
- Aumenta la cantidad de clases

- Puede ser innecesario para objetos simples

- Se necesita controlar los objetos obligatorios

Prototype

Diagrama UML



Problema

El patrón Prototype busca solucionar el problema de crear nuevos objetos que sean similares a otros existentes, sin necesidad de construirlos desde cero.

Cuando un objeto tiene muchos atributos o su creación es costosa y compleja, puede resultar ineficiente crear cada instancia manualmente. Además, en muchos casos se necesitan múltiples objetos con una estructura similar, cambiando solo algunos datos específicos.

En este ejemplo, un `documento` tiene varios atributos como título, autor, contenido y tipo. Crear cada documento desde cero puede ser repetitivo y propenso a errores.

Solución

El patrón **Prototype** resuelve este problema permitiendo crear nuevos objetos a partir de un objeto existente, llamado “**prototipo”**, mediante la clonación.

Cada clase que quiera funcionar como prototipo implementa una interfaz con un método de clonación. En este caso, la interfaz `Prototype` define **Prototype clonar();**

La clase `Documento` implementa este método y devuelve una copia de sí misma utilizando un constructor de copia.

El **`RegistroPrototipos`** actúa como un repositorio de prototipos, donde se almacenan diferentes documentos modelo que pueden clonar cuando se necesiten nuevas instancias.

Consecuencias

Ventajas

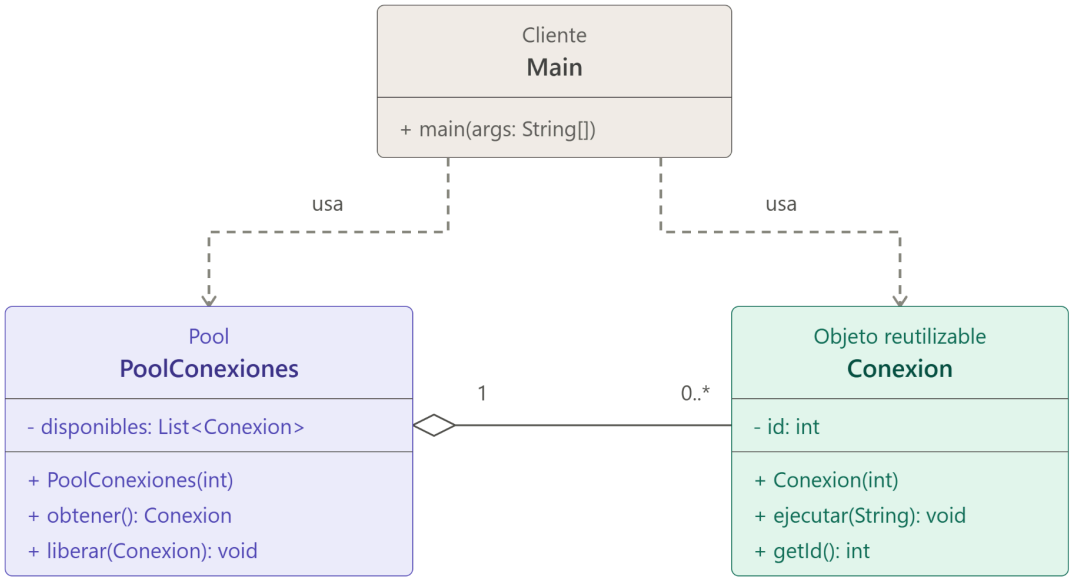
- Evita la creación de objetos desde cero.
- Reduce costos de creación.
- Facilita la creación de múltiples objetos similares.
- Permite una mayor flexibilidad.
- Encapsula el proceso de clonación.

Desventajas

- Puede ser complejo clonar objetos con referencias a otros objetos.
- Requiere que las clases implementen el método de clonación.
- No siempre es la mejor opción.

Objetc Pool

UML



Problema

Una aplicación necesita conectarse a la base de datos todo el tiempo. Crear una conexión es lento y costoso (hay que abrir el canal, autenticarse, reservar memoria).

Si cada vez que hace falta una conexión se crea una nueva y después se descarta:

- Se pierde mucho tiempo creando y destruyendo objetos todo el tiempo.

- Se gastan muchos recursos: con muchos usuarios a la vez, podrían crearse cientos de conexiones y saturar la base de datos.
- No hay control: nada limita cuántas conexiones existen.

Solución

El Object Pool crea los objetos una sola vez al principio y los reutiliza:

- `PoolConexiones` arma un conjunto fijo de conexiones al iniciar (en tu código, 2).
- Cuando alguien necesita una, la pide con `obtener()` y el pool se la presta.
- Cuando termina, la devuelve con `liberar()` y queda disponible para otro cliente.
- Si no queda ninguna libre, el pool lo avisa (en tu código devuelve `null`).

Así, en tu ejemplo, `c3` termina siendo la misma conexión que `c1`: se reutilizó en vez de crear una nueva.

Consecuencias

Lo bueno:

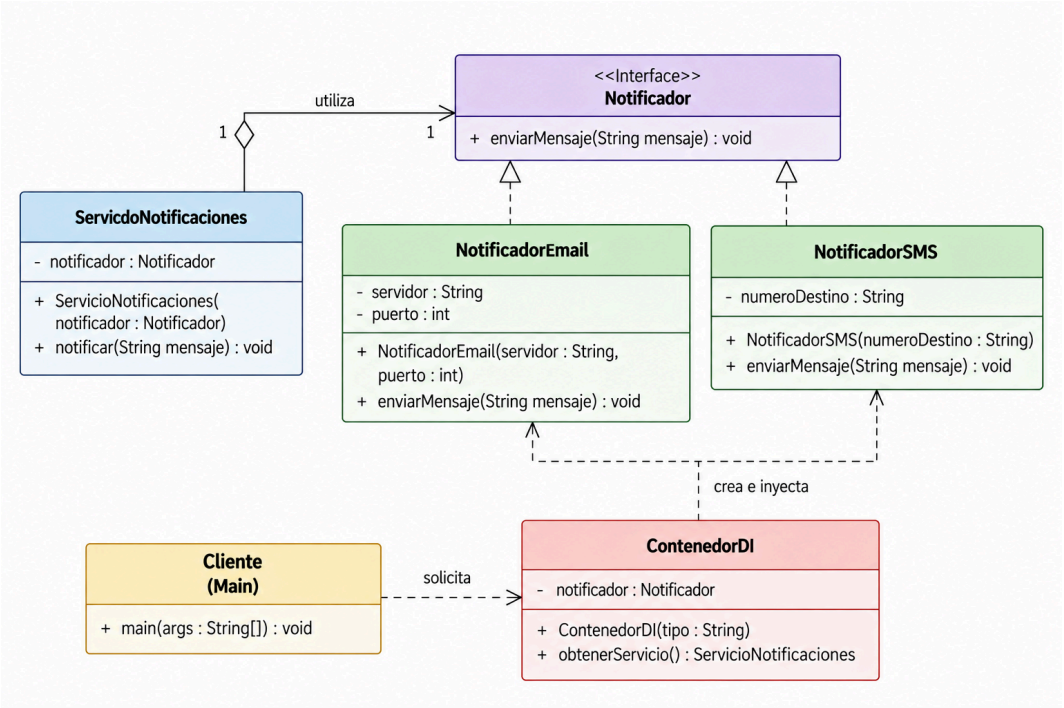
- Mejora el rendimiento: no se paga el costo de crear objetos una y otra vez.
- Se controla cuántos objetos existen, porque el pool tiene un máximo.
- Se aprovechan mejor los recursos del sistema.
- Es útil con recursos costosos, como conexiones, hilos o archivos.

Lo no tan bueno:

- Si todos los objetos están en uso, los clientes tienen que esperar o fallar.
- Hay que acordarse de devolver los objetos. Si alguien no llama a `liberar()`, el objeto se “pierde” y el pool se va quedando vacío.
- Un objeto reutilizado puede traer datos de su uso anterior, así que hay que dejarlo limpio antes de prestarlo de nuevo.
- Es más complejo que crear objetos directamente, y no vale la pena si crearlos es barato.
- Si varios clientes usan el pool a la vez (hilos), hay que sincronizarlo. Nuestra versión simple no lo hace.

Dependency Injection

Diagrama UML



Problema

Dependency Injection se utiliza para evitar que una clase cree directamente los objetos de los que depende.

Por ejemplo, imaginemos una clase `ServicioNotificacion` que necesita enviar mensajes.

Sin Dependency Injection podríamos hacer algo así:

```
public class ServicioNotificacion {

    private EmailNotificador notificador = new EmailNotificador();

    public void notificar(String mensaje) {

        notificador.enviar(mensaje);

    }

}
```

El problema es que `ServicioNotificacion` queda directamente acoplado a `EmailNotificador`.

Si después queremos utilizar SMS en lugar de correo electrónico, tendríamos que modificar la clase `ServicioNotificacion`.

Esto dificulta:

- Cambiar implementaciones.

- Reutilizar el código.
- Realizar pruebas.
- Mantener el sistema.
- Agregar nuevas formas de notificación.

Solución

Dependency Injection propone que una clase no cree sus propias dependencias, sino que las reciba desde afuera.

En este ejemplo participan:

- `Notificador`: interfaz que representa la dependencia.
- `EmailNotificador`: implementación para enviar mensajes por correo.
- `SMSNotificador`: implementación para enviar mensajes por SMS.
- `ServicioNotificacion`: clase que necesita un `Notificador`.
- `MainDependencyInjection`: crea los objetos e inyecta la dependencia.

En lugar de hacer:

```
private EmailNotificador notificador = new EmailNotificador();
```

La dependencia se recibe mediante el constructor:

```
public ServicioNotificacion(Notificador notificador) {

    this.notificador = notificador;

}
```

De esta forma, `ServicioNotificacion` no necesita saber qué implementación concreta está utilizando.

Puede trabajar con cualquier clase que implemente: Notificador

Consecuencias

Ventajas

- Reduce el acoplamiento
- Facilita cambiar implementaciones
- Facilita las pruebas
- Mejora la reutilización
- Facilita el mantenimiento
- Favorece el principio de inversión de dependencias

Desventajas

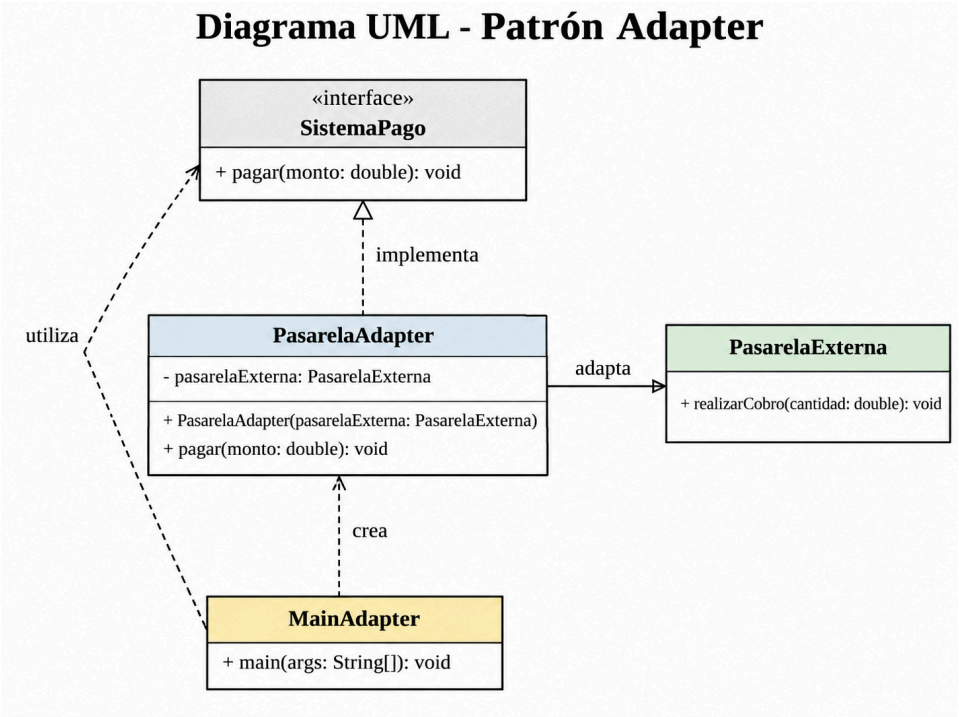
- Puede aumentar la cantidad de clases e interfaces
- La creación de objetos puede ser más compleja
- Puede ser excesivo en aplicaciones pequeñas

- Puede dificultar seguir el flujo del programa

Structural (Estructurales):

Adapter

Diagrama UML



Problema

El patrón **Adapter** busca solucionar el problema de incompatibilidad entre dos interfaces que necesitan trabajar juntas.

Puede ocurrir que tengamos una clase existente que realiza la función que necesitamos, pero sus métodos o su interfaz son diferentes a los que espera nuestro programa.

Por ejemplo, nuestra aplicación trabaja con una interfaz llamada **`SistemaPago`** que posee el método **void pagar(double monto);**

Sin embargo, tenemos una clase externa llamada **`PasarelaExterna`** que realiza pagos mediante otro método **void realizarCobro(double cantidad);**

Aunque ambas clases realizan prácticamente la misma función, el programa no puede utilizar directamente **`PasarelaExterna`** como un **`SistemaPago`** porque sus interfaces son diferentes.

Modificar la clase externa puede no ser posible o no ser conveniente, especialmente si pertenece a una librería externa.

Solución

En el patrón se crea una clase intermedia llamada "**Adapter"**, cuya función es hacer compatibles ambas interfaces.

En este ejemplo se crea la clase **PasarelaAdapter**

Esta clase implementa la interfaz **`SistemaPago`**, que es la interfaz que espera nuestra aplicación.

Internamente, el Adapter contiene una instancia de **`PasarelaExterna`.**

Cuando el programa llama a **pagar(monto);**

El Adapter transforma esa llamada en **pasarelaExterna.realizarCobro(monto);**

De esta manera, el código principal puede trabajar con la interfaz **`SistemaPago`** sin preocuparse por cómo funciona internamente la pasarela externa.

El Adapter actúa como un traductor entre dos interfaces incompatibles.

Consecuencias

Ventajas

-Permite reutilizar clases existentes.

-Evita modificar clases externas.

-Reduce el acoplamiento.

-Facilita integrar sistemas externos.

-Separa la lógica de adaptación.

-Aumenta la flexibilidad.

Desventajas

-Aumenta la cantidad de clases.

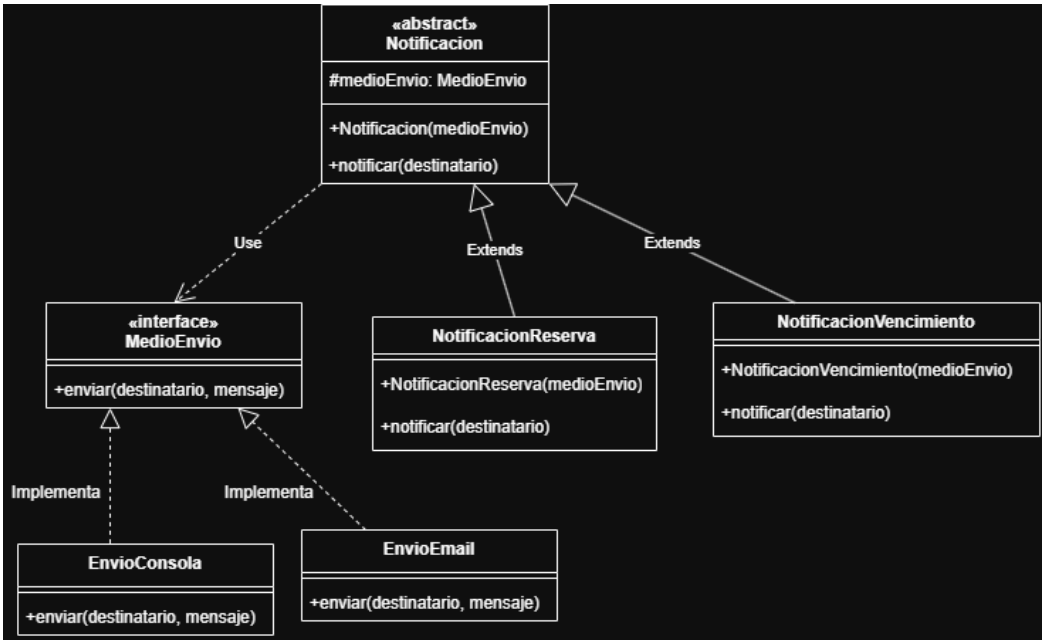
-Puede agregar complejidad.

-Puede haber muchos adapters.

-El Adapter debe conocer ambas interfaces.

Bridge

1. **Código Java funcionando**
2. **Diagrama UML**



3. **Problema - Solución - Consecuencias**
    a. **Problema:**
        - **En un sistema de gestión de biblioteca, es necesario enviar diferentes tipos de notificaciones a los socios, como avisos de reservas disponibles y recordatorios de vencimiento de préstamos. Además, las notificaciones pueden enviarse por distintos medios, como la consola o el correo electrónico.**

          Si los tipos de notificación y los medios de envío se implementan juntos en las mismas clases, la cantidad de clases y las dependencias aumentan a medida que se agregan nuevas funcionalidades. Por ejemplo, podríamos terminar creando una clase para las reservas por consola, otra para las reservas por correo, otra para los vencimientos por consola y otra para los vencimientos por correo.

          Esto dificulta el mantenimiento y la ampliación del sistema.

    b. **Solución:**
        - **El patrón Bridge separa el tipo de notificación del medio utilizado para enviarla.**
          Para ello, se define una abstracción Notificacion, que contiene una referencia a la interfaz MedioEnvio. Los distintos tipos de notificación heredan de la abstracción, mientras que los medios concretos, como EnvioConsola y EnvioEmail, implementan la interfaz de envío.

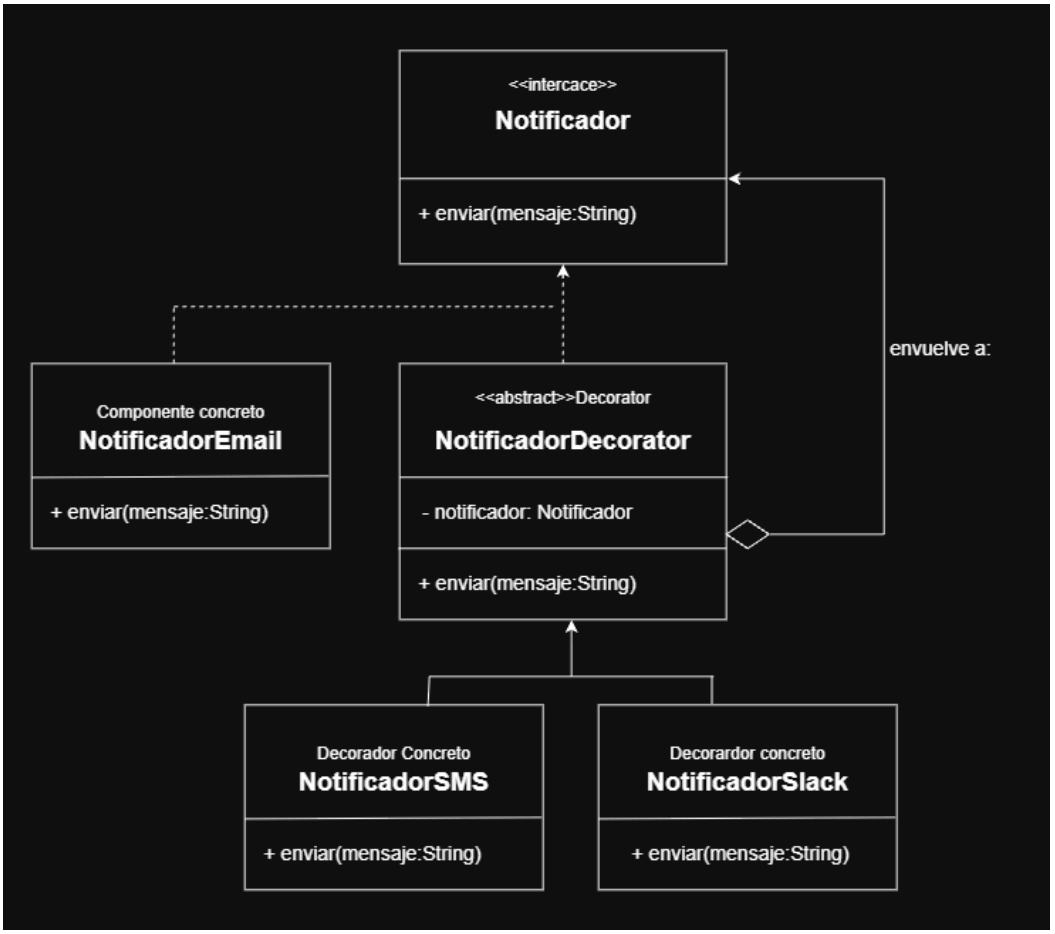
De esta manera, ambas partes pueden evolucionar independientemente y combinarse sin necesidad de crear una clase para cada combinación posible.

c. Consecuencias:

- **Ventajas:**
  - **Reduce el acoplamiento entre los tipos de notificación y los medios de envío.**
  - **Facilita la incorporación de nuevos tipos de notificación y nuevos medios.**
  - **Evita crear una clase específica para cada combinación.**
  - **Mejora el mantenimiento y la reutilización del código.**
- **Desventajas:**
  - **Aumenta la cantidad de interfaces y clases que hay que comprender y mantener.**
  - **Introduce una estructura más compleja que podría ser innecesaria en un sistema pequeño con pocas variantes.**
  - **Requiere identificar correctamente las dos dimensiones que deben evolucionar de forma independiente.**

Decorator

UML



Problema

Queremos agregar funcionalidades a un objeto y poder combinarlas como queramos, pero las opciones tradicionales fallan:

- **Con herencia** hay una explosión de subclases. Para combinar email, SMS y Slack se necesitaría EmailSMS, EmailSlack, EmailSMSSlack... y cada canal nuevo duplica las combinaciones.
- **Modificar la clase original** meter muchos condicionales (como if SMS… if Slack) dentro de NotificadorEmail la vuelve cada vez más grande y frágil.

Solución

El Decorator **envuelve** al objeto original con otro objeto que implementa la misma interfaz. El envoltorio delega en el objeto envuelto y agrega su propio comportamiento.

- **Sin tocar la clase original:** NotificadorEmail no cambia nunca. Cada funcionalidad nueva es una clase nueva.
- **Dinámico:** las capas se apilan en tiempo de ejecución, como en new NotificadorSlack(new NotificadorSMS(new NotificadorEmail())).

Ventajas

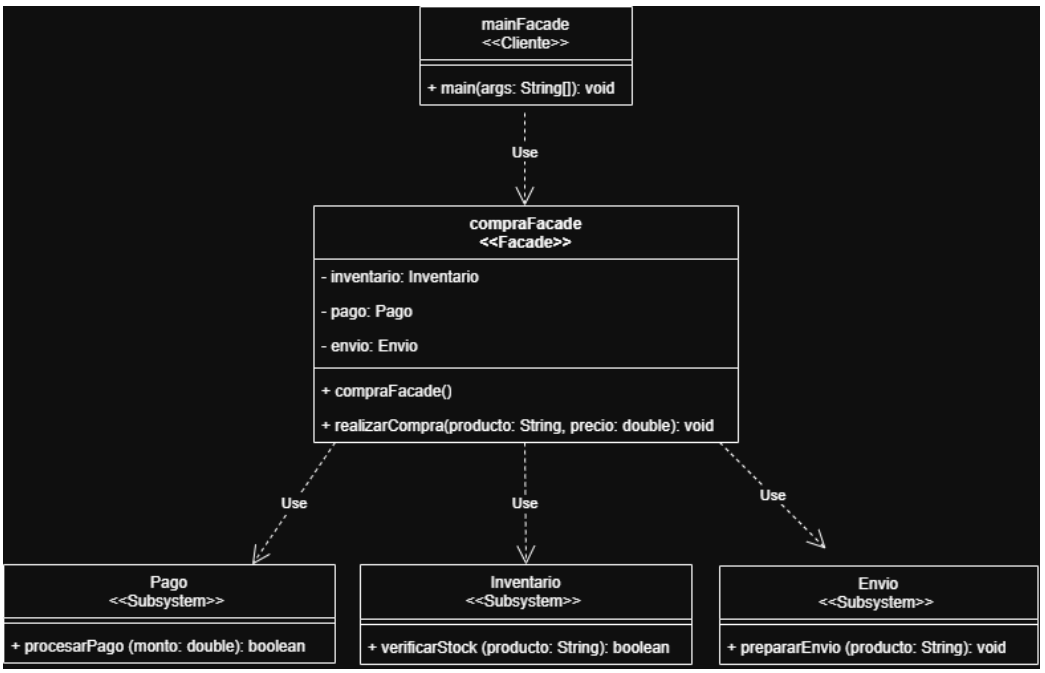
- Es más flexible que la herencia: se combinan funcionalidades en ejecución.
- Cada decorator tiene una sola responsabilidad (un canal), lo que facilita el mantenimiento.
- Evita clases gigantes llenas de condicionales.

Desventajas

- Genera muchas clases pequeñas y parecidas.
- Armar el objeto es más verboso (varios new anidados) y cuesta más leer.
- El orden de las capas puede importar.
- Es difícil quitar un decorator del medio de la cadena.

Facade

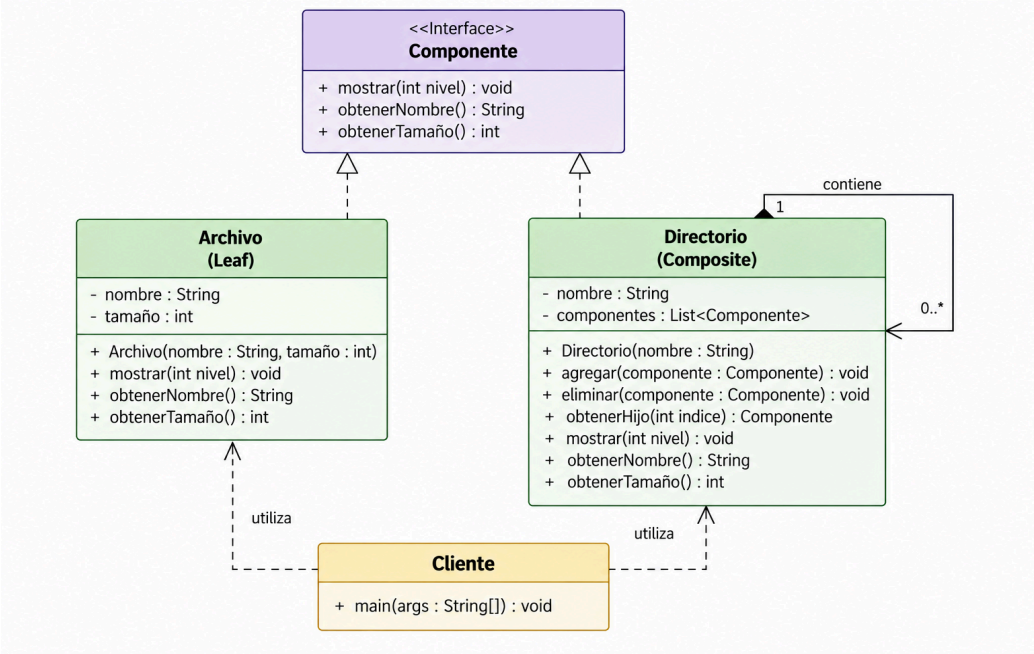
1. **Código Java funcionando**
2. **Diagrama UML**



3. **Problema - Solución - Consecuencias**
    a. **Problema:**
    Cuando una aplicación crece, el cliente (el código que usa los objetos) tiene que interactuar directamente con muchas clases complejas y acopladas al mismo tiempo (en este ejemplo: verificar stock, cobrar, coordinar envíos). Esto hace que el código del cliente sea difícil de leer, mantener y que cualquier cambio en los subsistemas rompa al cliente.
    b. **Solución:**
    Se coloca una clase intermediaria (la Fachada o compraFacade) que funciona como una "ventanilla única". En lugar de que el cliente hable con tres clases distintas, ahora habla únicamente con la fachada, y es la fachada la que se encarga de orquestar y ordenar a los subsistemas por detrás.
    c. **Consecuencias**
        - **Ventajas:**
            - **Código más limpio y fácil de usar:** El cliente queda súper simple (en lugar de 10 líneas de código con lógica dispersa, solo llama a un método como realizarCompra).
            - **Menor acoplamiento:** Si mañana cambias cómo funciona la clase Pago, solo modificas la fachada o el subsistema, pero el cliente (Main) ni se entera y no hay que tocarlo.
        - **Desventajas:**
            - Si la fachada crece demasiado y empieza a hacer de todo, puede convertirse en un objeto gigantesco y difícil de mantener (se le llama "objeto todopoderoso").

Composite

Diagrama UML



Problema

El patrón **Composite** se utiliza cuando necesitamos trabajar con objetos individuales y grupos de objetos de la misma manera.

Un ejemplo típico es un sistema de archivos:

- Un `Archivo` representa un elemento individual.
- Un `Directorio` puede contener archivos y también otros directorios.

Sin utilizar Composite, el programa tendría que comprobar constantemente si está trabajando con un archivo o con un directorio para saber qué operación realizar.

Esto puede generar muchos `if`, `else` o comprobaciones de tipo y hacer que el código sea más difícil de mantener.

Solución

El patrón **Composite** crea una interfaz común que utilizan tanto los objetos individuales como los objetos compuestos.

En este ejemplo participan:

- `Componente`: interfaz común.

- `Archivo`: representa un elemento individual o **Leaf**.
- `Directorio`: representa un objeto compuesto o **Composite**.
- `MainComposite`: actúa como cliente.

La interfaz `Componente` define operaciones comunes:

void mostrar(int nivel);

String obtenerNombre();

int obtenerTamanio();

Tanto `Archivó` como `Directorio` implementan esta interfaz.

La diferencia es que el Directorio puede almacenar una lista de otros objetos `Componente`.

Por lo tanto, puede contener: Directorio

↳ Archivo

↳ Archivo

↳Directorio

↳Archivo

El cliente puede trabajar con todos ellos mediante la misma interfaz.

Consecuencias

Ventajas

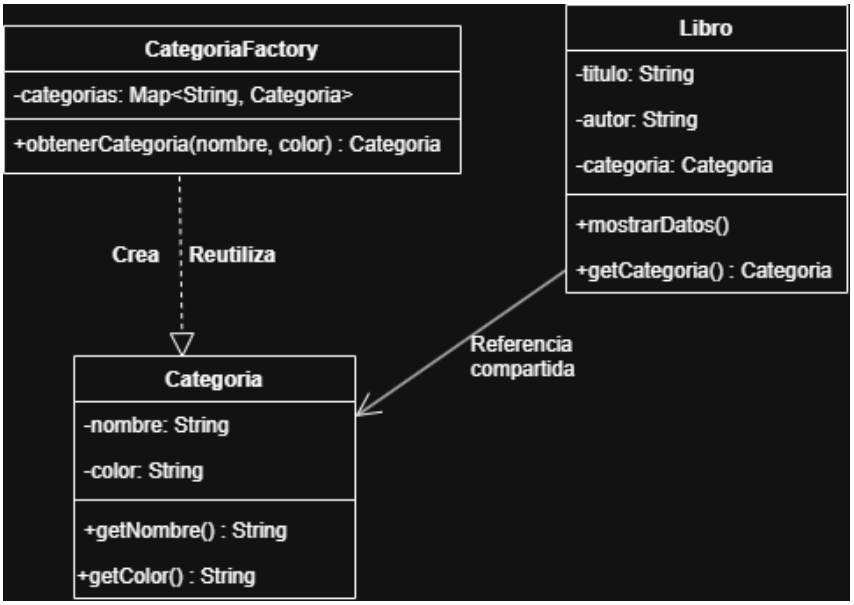
- Permite tratar objetos individuales y compuestos de la misma manera
- Facilita crear estructuras jerárquicas
- Simplifica el código cliente
- Permite estructuras recursivas
- Facilita agregar nuevos tipos de componentes
- Mejora la organización del código

Desventajas

- Puede hacer demasiado general la interfaz
- Puede ser difícil establecer restricciones
- Puede aumentar la complejidad en estructuras grandes
- Puede ser innecesario para estructuras simples

Flyweight

Diagrama UML



Problema:

- En un sistema de gestión de biblioteca existen muchos libros, y cada uno pertenece a una categoría (por ejemplo “Novela” o “Ciencia”), que tiene un nombre y un color. Si cada libro crea su propio objeto Categoría, se generan muchos objetos idénticos que guardan los mismos datos. Esto consume memoria innecesariamente y se vuelve ineficiente a medida que crece la cantidad de libros.

Solución:

- El patrón Flyweight permite compartir los objetos que se repiten en lugar de crear uno nuevo cada vez. En nuestro ejemplo, los datos comunes (nombre y color de la categoría) se separan de los datos propios de cada libro (título y autor). La clase CategoriaFactory guarda las categorías en un Map y, mediante obtenerCategoria(nombre, color), revisa si la categoría ya existe: si existe, la reutiliza, y si no, la crea y la guarda. De esta manera, todos los libros de una misma categoría comparten una única referencia al mismo objeto Categoría.

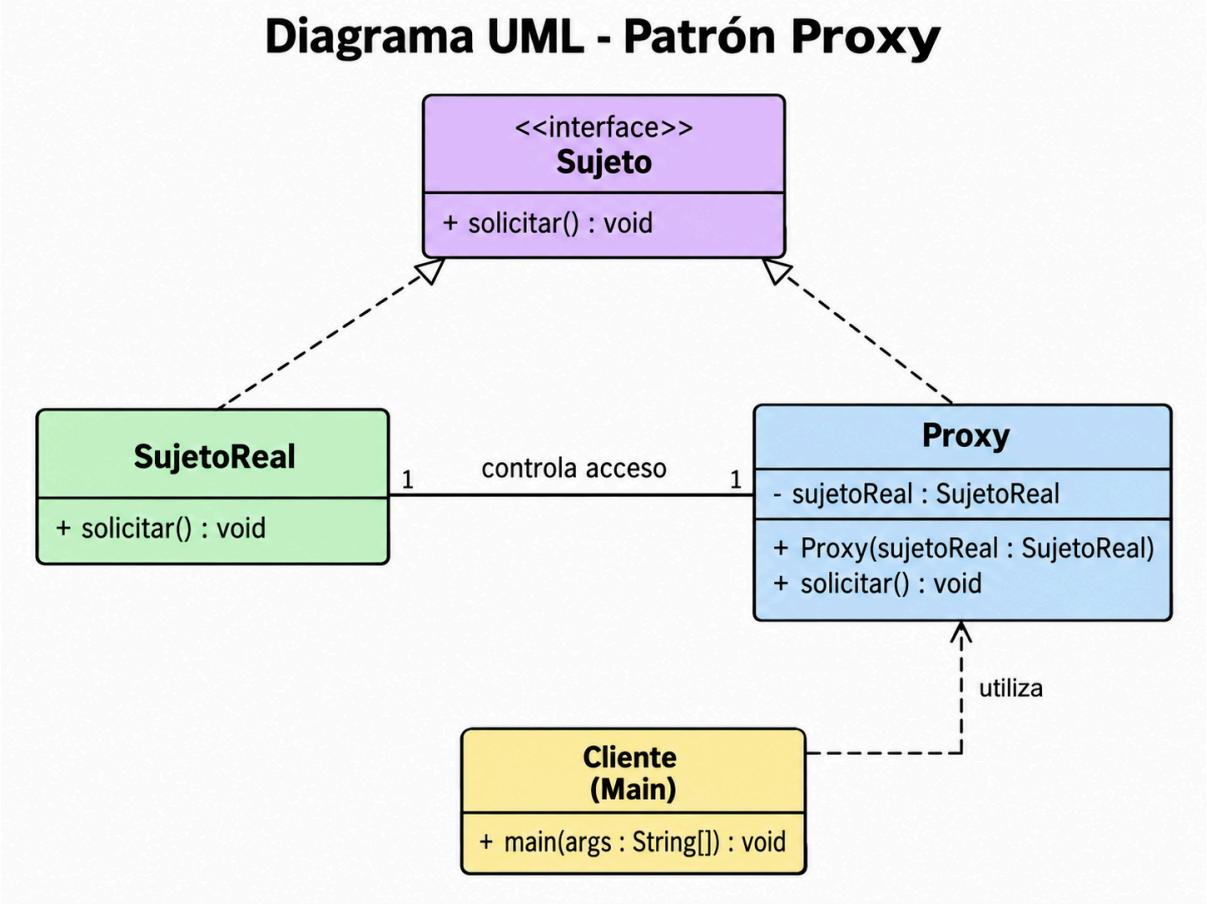
Consecuencias:

- **Ventajas**:
  - Reduce el consumo de memoria al evitar objetos duplicados.
  - Centraliza la creación de las categorías en la fábrica.
  - Mejora el rendimiento cuando hay una gran cantidad de objetos similares.
  - Permite compartir los datos comunes entre muchos objetos.
- **Desventajas**:
  - Agrega clases (la fábrica) y vuelve el diseño más complejo.
  - Hay que separar correctamente los datos compartidos de los propios de cada objeto.
  - Los objetos compartidos deben ser inmutables, porque un cambio en uno afectaría a todos los libros que lo usan.

- Puede ser innecesario si hay pocos objetos.

Proxy

Diagrama UML



Problema

El patrón **Proxy** se utiliza cuando necesitamos controlar el acceso a un objeto sin que el cliente tenga que interactuar directamente con él.

Por ejemplo, imaginemos un sistema que permite acceder a un recurso mediante una clase llamada **`SujetoReal`**.

En algunos casos, no queremos que cualquier parte del programa utilice directamente ese objeto, ya que puede ser necesario verificar el acceso, registrar las operaciones o controlar cuándo se ejecuta una acción.

Sin Proxy, el cliente tendría que encargarse de realizar esas verificaciones antes de utilizar el objeto real.

Esto genera problemas porque aumenta el acoplamiento y mezcla la lógica de acceso con la lógica principal del programa.

Solución

El patrón **Proxy** crea una clase intermediaria que controla el acceso al objeto real.

En este ejemplo participan:

- **`Sujeto`**: interfaz común que define la operación disponible.
- **`SujetoReal`**: clase que realiza la operación real.
- **`Proxy`**: clase intermediaria que controla el acceso a `SujetoReal`.
- **`Main`**: cliente que utiliza el Proxy.

Consecuencias

Ventajas

- Controla el acceso a los objetos
- Reduce el acoplamiento
- Permite agregar seguridad
- Permite registrar operaciones
- Facilita la carga diferida
- Separa responsabilidades

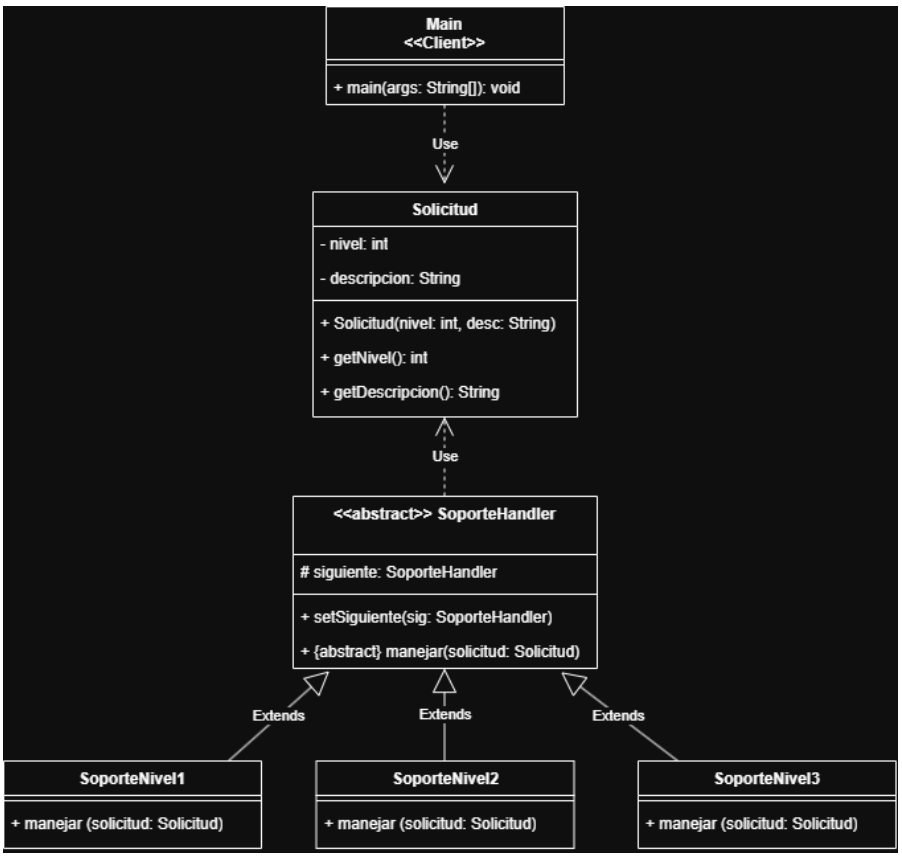
Desventajas

- Aumenta la cantidad de clases
- Puede agregar complejidad
- Puede afectar el rendimiento
- Puede dificultar el seguimiento de las operaciones

Behavioral (Conductuales):

Chain of Responsibility

1. **Código Java funcionando**
2. **Diagrama UML**

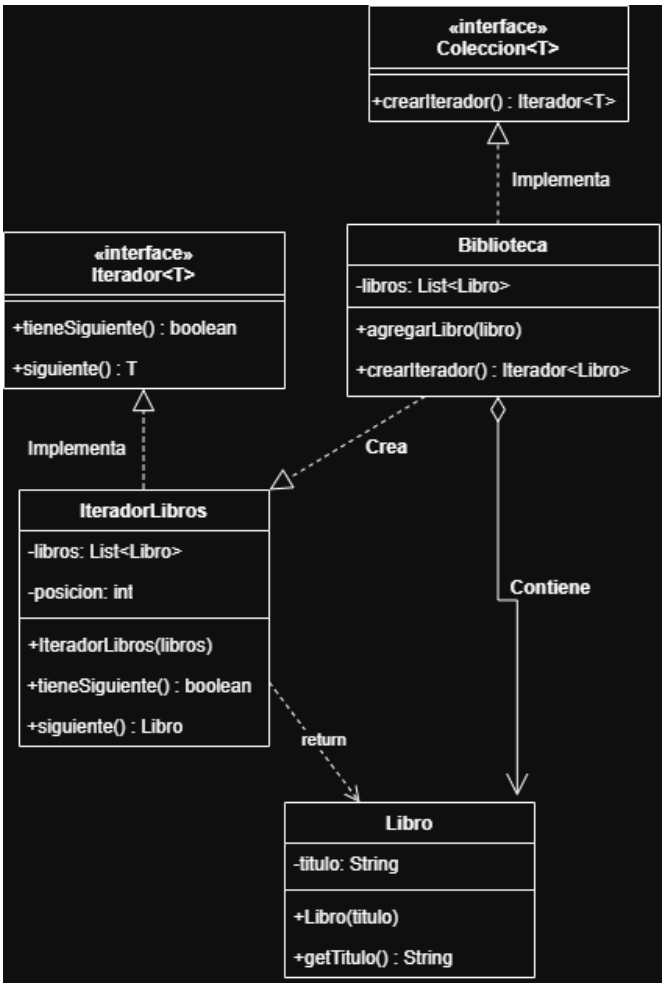


3. **Problema - Solución - Consecuencia**
    a. **Problema:**
    Cuando tienes una petición o solicitud que puede ser procesada por diferentes objetos (por ejemplo, distintos niveles de soporte técnico), si usas un enfoque tradicional con condicionales gigantes (if/else if), el código se vuelve rígido, difícil de mantener y viola el principio de diseño de "responsabilidad única". Cada vez que agregues un nuevo nivel de soporte, tendrías que modificar una clase central rompiendo código existente.
    b. **Solución:**
    Implementar el patrón Chain of Responsibility. Consiste en crear una cadena de objetos receptores (los manejadores) donde cada uno tiene una referencia al siguiente. Cuando llega una solicitud, el primer objeto evalúa si puede resolverla; si no puede, se la pasa automáticamente al siguiente eslabón de la cadena sin que el cliente tenga que saber quién la terminó resolviendo.
    c. **Consecuencia:**
        - **Ventajas:**
            - **Desacoplamiento:** El objeto que envía la solicitud (el cliente) no necesita saber qué objeto exacto la va a resolver.
            - **Flexibilidad:** Puedes agregar, quitar o reordenar los eslabones de la cadena fácilmente modificando el Main (o mediante configuración), sin tocar el código interno de los demás manejadores.
            - **Principio de responsabilidad única:** Cada clase manejadora se encarga exclusivamente de su propia lógica de decisión.
        - **Desventajas:**
            - **Sin garantía de atención:** Si la cadena está mal armada o ningún objeto maneja la solicitud, esta puede llegar al final de la cadena y perderse o no resolverse.

- **Rendimiento:** Si la cadena es muy larga, puede haber un pequeño costo de procesamiento al recorrer los eslabones hasta encontrar al responsable.

Iterator

1. **Código Java funcionando**
2. **Diagrama UML**



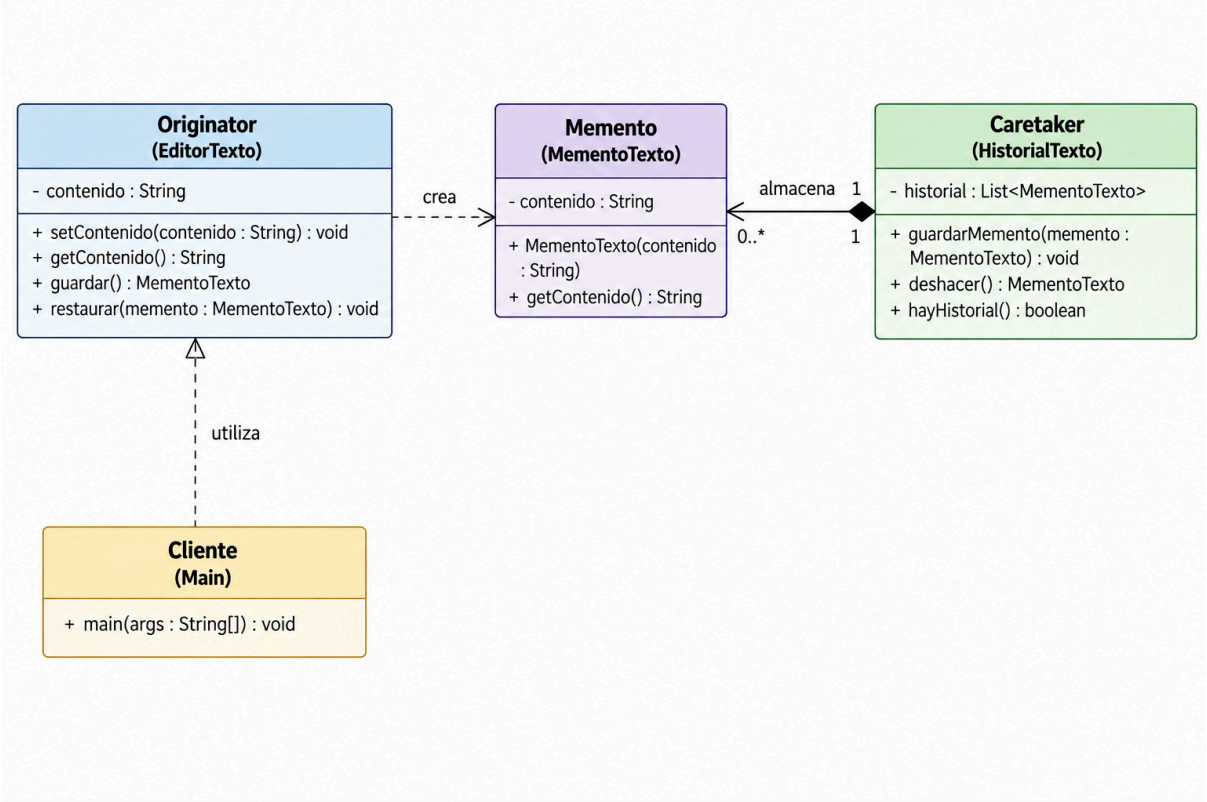
3. **Problema - Solución - Consecuencia**
   a. **Problema:**
      - **En un sistema de gestión de biblioteca, necesitamos recorrer los libros almacenados para mostrar sus títulos, consultar sus datos o realizar otras operaciones.**
        Si cada parte del programa debe conocer cómo está organizada la colección y controlar directamente sus posiciones, el código queda acoplado a la estructura interna de los datos. Esto dificulta los cambios y puede llevar a repetir la misma lógica de recorrido en distintas partes del sistema.
   b. **Solución:**
      - **El patrón Iterator permite recorrer los elementos de una colección mediante un objeto independiente llamado iterador, sin exponer directamente la lógica de recorrido al código que utiliza la colección. En nuestro ejemplo, la clase Biblioteca almacena los libros y proporciona un iterador mediante crearIterador(). La clase IteradorLibros implementa las operaciones tieneSiguiente() y**

siguiente(), que permiten comprobar si quedan elementos y obtenerlos uno por uno.
De esta manera, el código que utiliza la biblioteca puede recorrer sus libros sin administrar directamente los índices de la lista.

c. **Consecuencia:**
   - **Ventajas:**
     - **Separa la lógica de recorrido de la estructura interna de la colección.**
     - **Permite recorrer los elementos sin exponer cómo se almacenan.**
     - **Facilita reutilizar el mecanismo de recorrido en distintas partes del programa.**
     - **Permite modificar la implementación del iterador sin cambiar necesariamente el código que lo utiliza.**
   - **Desventajas:**
     - **Agrega clases e interfaces, aumentando la complejidad del diseño.**
     - **Si solo necesitamos recorrer una lista sencilla, implementar un iterador propio puede ser innecesario.**
     - **Hay que controlar correctamente los límites del recorrido para evitar errores cuando no quedan elementos.**

Memento

Diagrama UML



Problema

El patrón Memento se utiliza cuando necesitamos guardar el estado de un objeto para poder restaurarlo posteriormente. Un ejemplo típico es un editor de texto con la función **deshacer**.

Solución

El patrón **Memento** permite guardar una copia del estado interno de un objeto sin exponer directamente sus atributos.

En este ejemplo participan:

- **`EditorTexto`**: es el **Originator**. Contiene el estado actual del texto.
- **`MementoEditor`**: es el **Memento**. Guarda una copia del contenido del editor.
- **`Historial`**: es el **Caretaker**. Guarda los distintos Mementos.
- **`MainMemento`**: utiliza las clases y demuestra cómo guardar y restaurar estados.

Cuando queremos guardar el estado actual:

editor.guardar();

se crea un nuevo **`MementoEditor`**.

El historial almacena ese Memento:

historial.guardar(editor.guardar());

Luego, si queremos volver a un estado anterior:

editor.restaurar(historial.deshacer());

El objeto **`EditorTexto`** recupera el contenido que tenía anteriormente.

Consecuencias

Ventajas

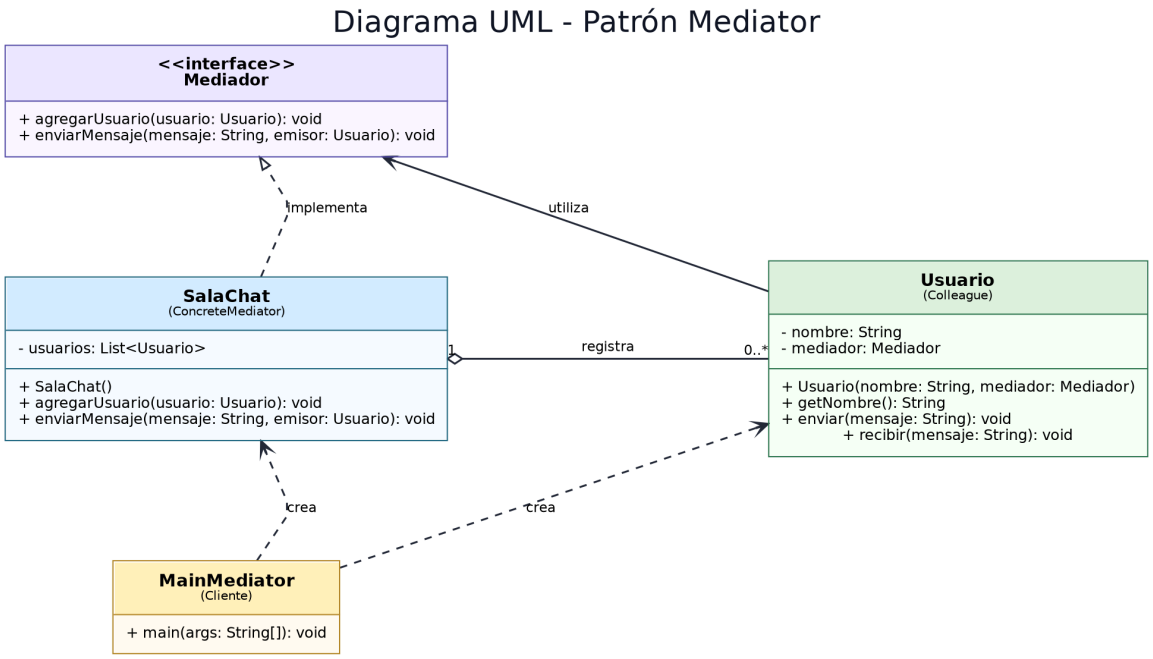
- Permite implementar deshacer:
- Mantiene el encapsulamiento:
- Separa las responsabilidades:
- Facilita guardar diferentes versiones:
- Evita que el historial conozca los detalles internos

Desventajas

- Puede consumir mucha memoria
- Puede generar muchos objetos
- La administración del historial puede ser compleja
- No siempre es necesario

Mediator

Diagrama UML



Problema

El patrón Mediator se utiliza cuando varios objetos necesitan comunicarse entre sí, pero queremos evitar que dependan directamente unos de otros.

Solución

El patrón Mediator crea un objeto intermediario que se encarga de coordinar la comunicación entre los demás objetos.

Consecuencias

Ventajas

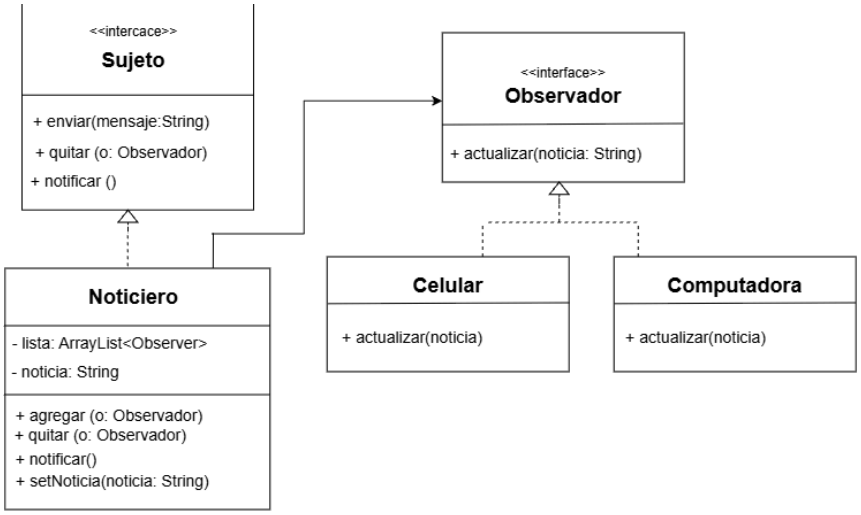
- Reduce el acoplamiento
- Centraliza la comunicación
- Facilita el mantenimiento
- Permite agregar nuevos participantes
- Mejora la reutilización
- Facilita la organización del código

Desventajas

- El Mediator puede volverse demasiado complejo
- Puede convertirse en un punto central de dependencia
- Puede ser innecesario para sistemas simples
- Puede dificultar el mantenimiento del mediador

Observer

UML



Problema

Hay objetos que dependen del estado de otro objeto y necesitan enterarse cuando cambia. En nuestro ejemplo, el Noticiero tiene una noticia, y el Celular y la Computadora quieren recibirla apenas cambia.

Solución

El patrón Observer separa los roles en dos:

- El Sujeto (Noticiero) guarda una lista de observadores. Cuando cambia su estado, recorre la lista y le avisa a cada uno llamando a actualizar().
- Los Observadores (Celular, Computadora) se registran con agregar() y se dan de baja con quitar().

El sujeto no sabe qué tipo de objetos lo escuchan, solo sabe que implementan la interfaz Observador. Así los avisos son automáticos y se pueden sumar o sacar observadores sin tocar el código del sujeto.

Ventajas

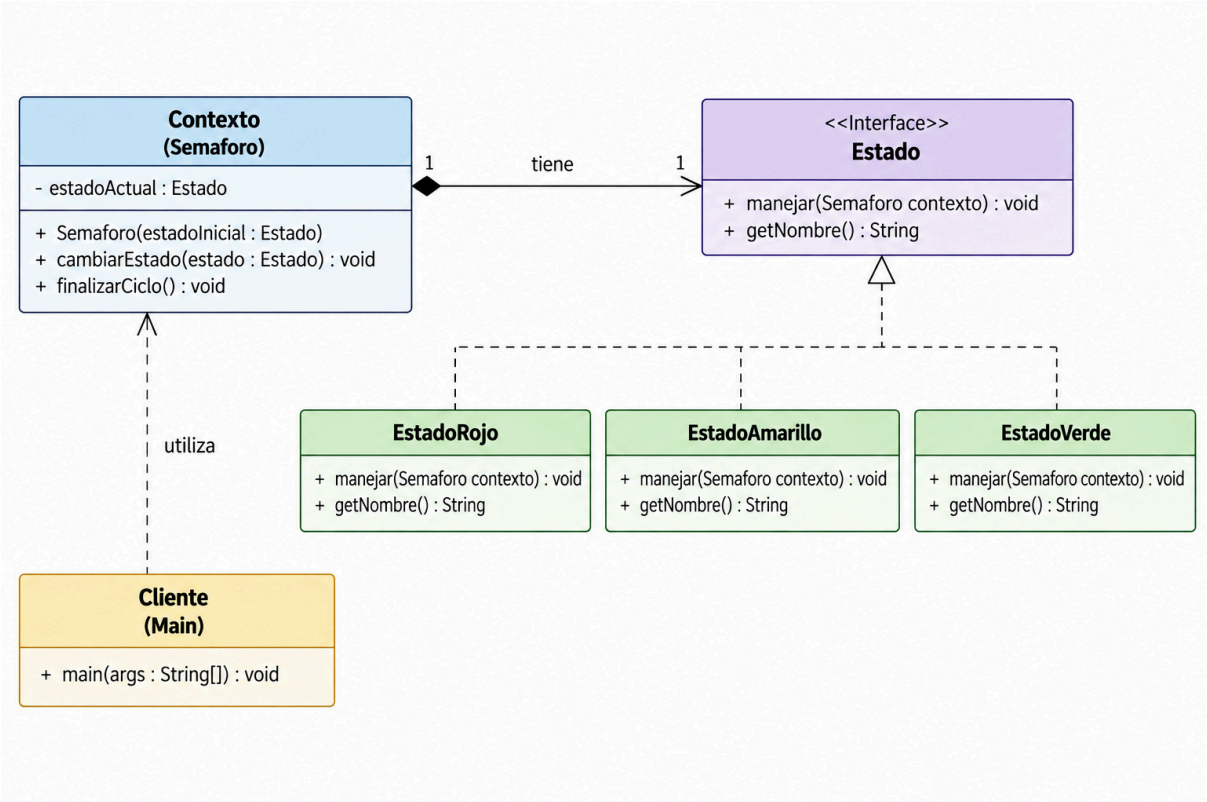
- Hay bajo acoplamiento: el sujeto solo conoce la interfaz Observador, no las clases concretas.
- Se pueden agregar o quitar observadores en cualquier momento.
- Las notificaciones son automáticas, sin necesidad de estar consultando.
- Un mismo sujeto puede avisar a muchos observadores distintos, cada uno reaccionando a su manera.

Desventajas

- Si hay muchos observadores, notificar a todos puede volverse lento.
- Un observador que no se quita de la lista sigue recibiendo avisos aunque ya no se use (puede generar errores o gastar memoria).
- Es difícil seguir el flujo del programa, porque un solo cambio dispara muchas acciones que no se ven a simple vista.

State

Diagrama UML



Problema

El patrón **State** se utiliza cuando un objeto puede comportarse de manera diferente dependiendo del estado en el que se encuentra.

Por ejemplo, un reproductor de música puede encontrarse en distintos estados:

- Reproduciendo
- Pausado
- Detenido

Solución

El patrón **State** separa el comportamiento de cada estado en una clase diferente.

Todas las clases de estado implementan una misma interfaz.

En este ejemplo participan:

- **`EstadoReproductor`**: interfaz que define el comportamiento de los estados.
- **`EstadoReproduciendo`**: representa el estado reproduciendo.
- **`EstadoPausado`**: representa el estado pausado.
- **`EstadoDetenido`**: representa el estado detenido.
- **`Reproductor`**: es el contexto y mantiene el estado actual.
- **`MainState`**: demuestra el funcionamiento del patrón.

El **`Reproductor`** no necesita contener grandes bloques de condicionales.

Simplemente delega el comportamiento al estado actual:

estado.actualizarEstado(this);

Cada estado decide qué debe hacer y a qué nuevo estado debe cambiar el reproductor.

Ventajas

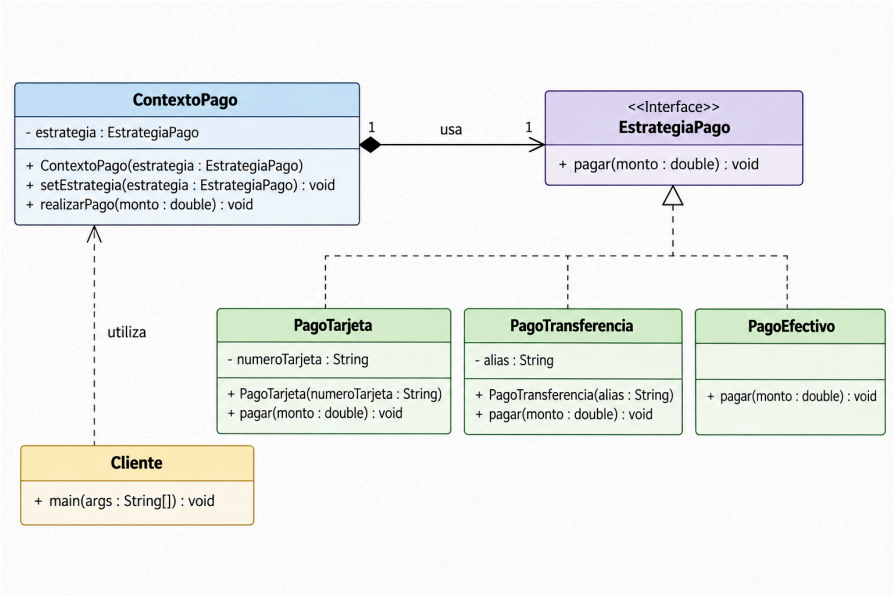
- Evita grandes bloques de condicionales
- Organiza mejor los estados:
- Facilita agregar nuevos estados
- Permite cambiar el comportamiento en tiempo de ejecución
- Mejora el mantenimiento
- Separa el estado del contexto

Desventajas

- Aumenta la cantidad de clases
- Puede ser innecesario para pocos estados
- Las transiciones pueden volverse complejaS
- Los estados deben conocer el contexto

Strategy

Diagrama UML



Problema

El patrón **Strategy** se utiliza cuando un objeto puede realizar una misma tarea de distintas maneras y queremos poder cambiar el algoritmo o comportamiento sin modificar la clase principal.

Por ejemplo, un sistema de pagos puede permitir diferentes formas de pago:

- Tarjeta de crédito
- PayPal
- Transferencia bancaria

Sin Strategy, podríamos terminar utilizando muchos `if, else o switch` dentro de una misma clase para decidir qué forma de pago usar.

Esto hace que el código sea más difícil de mantener y ampliar.

Solución

El patrón Strategy separa cada algoritmo o comportamiento en una clase diferente.

Todas estas clases implementan una misma interfaz.

En este ejemplo:

- **`EstrategiaPago`** define la operación `pagar()`.
- **`PagoTarjeta`** implementa el pago con tarjeta.
- **`PagoPayPal`** implementa el pago con PayPal.
- **`ContextoPago`** utiliza una estrategia de pago.
- **`MainStrategy`** selecciona qué estrategia utilizar.

La clase **`ContextoPago`** no necesita saber cómo funciona cada método de pago.

Simplemente ejecuta **estrategiaPago.pagar(monto);**

De esta manera, el comportamiento puede cambiarse en tiempo de ejecución.

Consecuencias

Ventajas

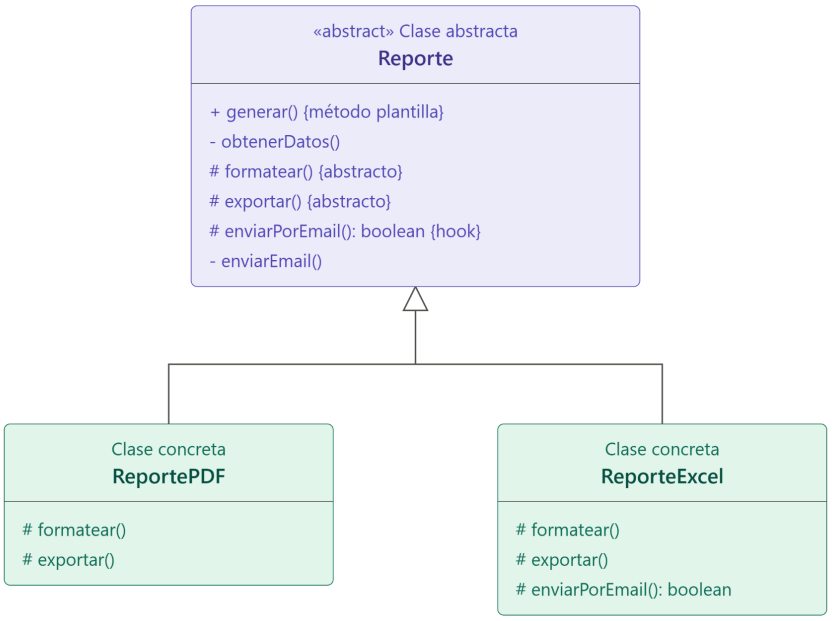
- Evita muchos condicionales
- Permite cambiar el comportamiento en tiempo de ejecución
- Facilita agregar nuevas estrategias
- Separa los algoritmos de la clase principal
- Mejora el mantenimiento
- Cumple con el principio abierto/cerrado

Desventajas

- Aumenta la cantidad de clases
- El cliente debe conocer las estrategias disponibles
- Puede ser innecesario para pocos comportamientos
- Puede haber muchas clases pequeñas

Template Method

UML



Problema

Tenemos reportes de ventas en PDF y en Excel. Los dos siguen **los mismos pasos en el mismo orden**: obtener datos, formatear, exportar y, a veces, enviar por email. Lo único que cambia es cómo se formatea y se exporta.

Si cada clase escribe todo el algoritmo por su cuenta:

- **Se repite código:** el paso de obtener los datos queda copiado en cada clase.
- **Es fácil que algo se rompa:** si cambia el orden de los pasos o un paso común, hay que modificar todas las clases, y alguna se puede olvidar o quedar distinta.

Solución

Se pone el **esqueleto del algoritmo en una sola clase abstracta**, dentro de un método plantilla.

- En `Reporte`, el método `generar()` define el orden de los pasos y es `final`, así que las subclases no lo pueden cambiar.
- Los pasos **iguales para todos** (`obtenerDatos()`) se escriben una sola vez en `Reporte`.
- Los pasos que **cambian** (`formatear()` y `exportar()`) son abstractos: cada subclase está obligada a completarlos.
- Los pasos **opcionales** son los *hooks* (`enviarPorEmail()`): tienen un comportamiento por defecto y la subclase puede cambiarlo si quiere, como hace `ReporteExcel`.

Es como una receta: el orden ya está escrito y cada subclase solo completa los ingredientes que le tocan. Además, quien manda es la clase base: ella llama a los métodos de las subclases, y no al revés.

Consecuencias

Lo bueno:

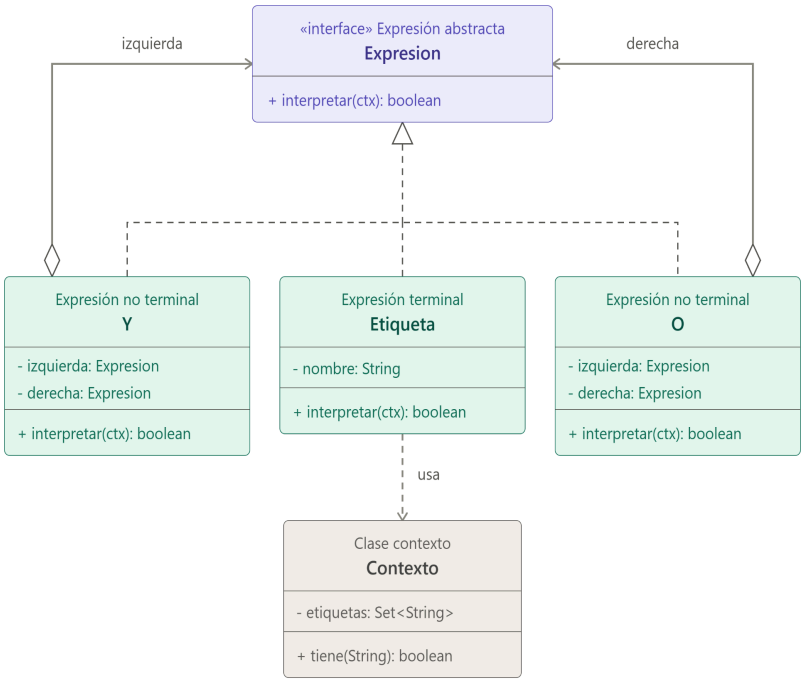
- No hay código repetido: lo común vive en un solo lugar.
- El orden de los pasos está controlado, porque `generar()` es `final`.
- Para agregar un formato nuevo (por ejemplo `ReporteCSV`) alcanza con crear una subclase, sin tocar las otras.
- Los hooks dan flexibilidad sin romper la estructura.

Lo no tan bueno:

- Usa herencia, así que las subclases quedan muy atadas a la clase base. Si ella cambia, puede afectar a todas.
- La estructura es fija y no se puede cambiar mientras el programa corre. Además, en Java una clase solo puede tener una superclase.
- Si la plantilla tiene muchos pasos, cada subclase tiene que implementar muchos métodos.
- Para entender el flujo completo hay que mirar la clase base y las subclases juntas.

Interpreter

UML



Problema

Una tienda tiene reglas de descuento como `VIP Y (CUPON O ESTUDIANTE)`. Esas reglas cambian seguido: hoy es una, mañana otra, y siempre son combinaciones de las mismas condiciones.

Si lo resolvemos con `if` anidados dentro del código:

- **Cada regla nueva obliga a modificar el código** y volver a compilar.
- **Se vuelve ilegible:** con varias condiciones combinadas, los `if` se enredan.
- **No se puede reutilizar:** no hay forma de armar o combinar reglas de manera flexible, porque cada una está “pegada” al código.

Solución

El Interpreter trata las reglas como un **pequeño lenguaje** y crea **una clase por cada pieza** de ese lenguaje:

- **Piezas simples** (expresiones terminales): `Etiqueta`, que pregunta “¿el cliente tiene VIP?”.
- **Piezas que combinan** (expresiones no terminales): `Y` y `O`, que guardan otras dos expresiones adentro.
- Todas implementan la misma interfaz, `Expresion`, con el método `interpretar()`.

Una regla queda armada como un **árbol**. Al llamar `interpretar()` en la raíz, cada nodo le pregunta a sus hijos, hasta llegar a las hojas, que consultan al `Contexto` (los datos del cliente). Ese paso por el árbol da el resultado final (`true` o `false`).

Para sumar algo nuevo (por ejemplo, un NO), se crea una clase nueva y no se toca ninguna de las que ya funcionaban.

Aclaración: armar el árbol (en tu código lo hace `Main` a mano) no es parte del patrón. El patrón se ocupa de cómo se representa y se interpreta la regla.

Consecuencias

Lo bueno:

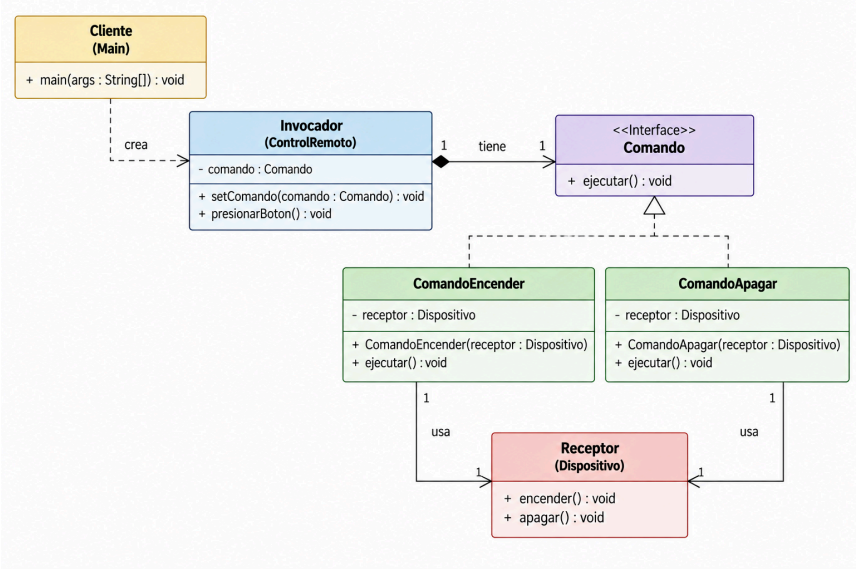
- Es fácil de ampliar: una regla nueva es una clase nueva.
- Las reglas se combinan y se reutilizan como piezas de rompecabezas.
- No hay `if` gigantes: cada clase hace una sola cosa.
- Para gramáticas simples, es sencillo de implementar y de entender.

Lo no tan bueno:

- Si el lenguaje crece, se multiplican las clases y se vuelve difícil de mantener.
- Es más lento: crea muchos objetos y recorre el árbol de forma recursiva.
- No incluye el *parser*, así que alguien tiene que armar el árbol (a mano o con otro código).
- Para lenguajes grandes o complejos conviene usar otras herramientas, como generadores de parsers.

Command

Diagrama UML



Problema

El patrón **Command** se utiliza cuando queremos representar una acción o solicitud como un objeto independiente.

Por ejemplo, imaginemos un control remoto que puede:

- Encender un dispositivo.
- Apagar un dispositivo.

Sin utilizar Command, el `ControlRemoto` tendría que conocer directamente los métodos del dispositivo:

```
dispositivo.encender();

dispositivo.apagar();
```

Esto genera un fuerte acoplamiento entre ambas clases.

Si posteriormente queremos agregar nuevas acciones o utilizar otros dispositivos, sería necesario modificar el código del control remoto.

Solución

El patrón **Command** encapsula cada acción dentro de una clase independiente.

En este ejemplo participan:

- `Comando`: interfaz común para todos los comandos.
- `ComandoEncender`: comando que enciende el dispositivo.
- `ComandoApagar`: comando que apaga el dispositivo.
- `Dispositivo`: es el **Receiver**, quien realiza realmente las acciones.
- `ControlRemoto`: es el **Invoker**, encargado de ejecutar los comandos.
- `MainCommand`: es el cliente que crea y relaciona los objetos.

La interfaz `Comando` define:

```
void ejecutar();
```

Cada comando implementa este método de manera diferente.

Por ejemplo:

```
public void ejecutar() {

   receptor.encender();

}
```

El `ControlRemoto` no necesita saber exactamente qué acción se está ejecutando.

Simplemente llama a: comando.ejecutar();

Consecuencias

Ventajas

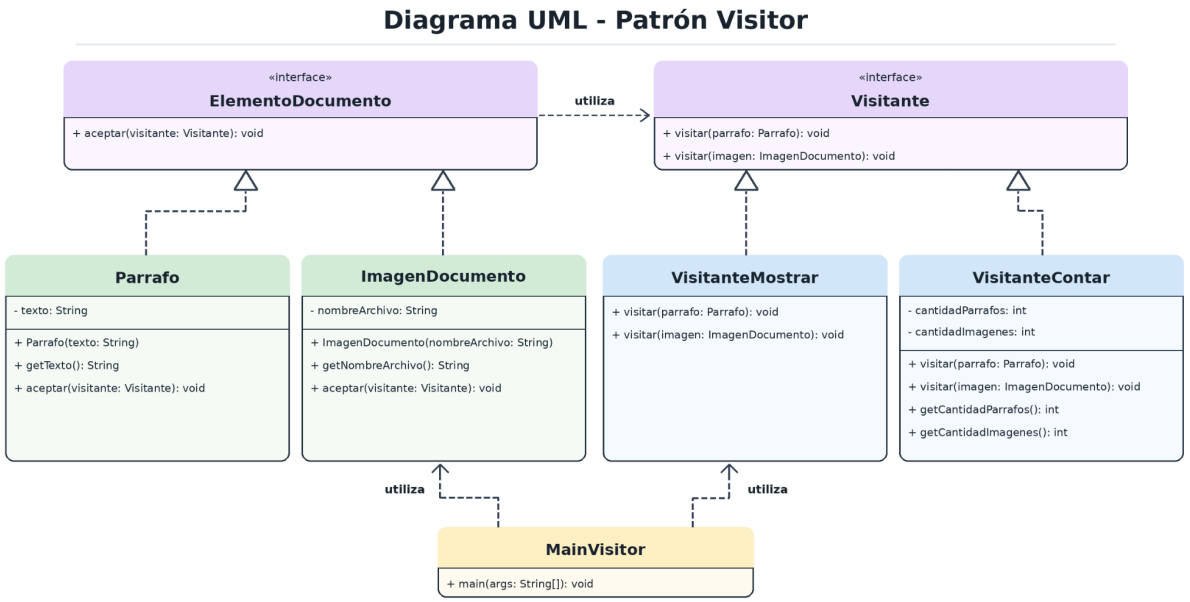
- Desacopla al emisor del receptor
- Facilita agregar nuevos comandos
- Permite reutilizar comandos
- Facilita implementar deshacer
- Permite crear historiales o colas
- Mejora la organización del código

Desventajas

- Aumenta la cantidad de clases
- Puede agregar complejidad
- Puede generar muchos comandos pequeños
- Requiere organizar bien las responsabilidades

Visitor

Diagrama UML



Problema

El patrón **Visitor** se utiliza cuando necesitamos realizar diferentes operaciones sobre un conjunto de objetos, pero queremos evitar modificar sus clases cada vez que agregamos una nueva funcionalidad.

Por ejemplo, imaginemos un documento que contiene diferentes elementos:

- Párrafos de texto.
- Imágenes.

Nuestro programa necesita mostrar estos elementos y también contar cuántos párrafos e imágenes existen.

Solución

El patrón **Visitor** separa las operaciones de los objetos sobre los que se realizan.

Para ello, utiliza una interfaz llamada **`Visitante`**, donde se definen los métodos para trabajar con cada tipo de elemento.

Consecuencias

Ventajas

- Permite agregar nuevas operaciones fácilmente
- Separa las responsabilidades
- Facilita el mantenimiento
- Permite trabajar con diferentes tipos de objetos
- Favorece el principio abierto/cerrado
- Permite reutilizar operaciones

Desventajas

- Aumenta la cantidad de clases
- Dificulta agregar nuevos tipos de elementos
- Puede aumentar el acoplamiento
- Puede requerir acceso a datos internos
- Puede ser innecesario en sistemas pequeños